Thala Chainlink Oracle

Security Assessment

May 13th, 2025 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope 2

General Findings **3**

OS-TCO-SUG-00 | Incorrect Price Negativity Check 4

Appendices

Vulnerability Rating Scale **5**

Procedure **6**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Thala Labs engaged OtterSec to assess the `chainlink-oracle` program. This assessment was conducted between April 20th and May 2nd, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 1 finding throughout this audit engagement.

In particular, we recommended updating the check to properly identify negative `i192` values (OS-TCO-SUG-00).

Scope

The source code was delivered to us in a Git repository at https://github.com/ThalaLabs/thala-modules. This audit was performed against PR#1015.

A brief description of the program is as follows:

Name	Description
chainlink-oracle PR#1015	The PR adds support for chainlink oracles, manages Chainlink price feed configurations, and retrieves asset prices.

© 2025 Otter Audits LLC. All Rights Reserved.

02 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-TCO-SUG-00	The current implementation of the price negativity check in `parse_chainlink_price` improperly identifies negative `i192` values.

© 2025 Otter Audits LLC. All Rights Reserved.

Incorrect Price Negativity Check

OS-TCO-SUG-00

Description

`chainlink_oracle::parse_chainlink_price` checks if a Chainlink `i192`-encoded price is negative by comparing it to `(1 << 192)`, but this is incorrect. In a signed 192-bit integer, the sign bit is at position 191, so the correct check should be `price >= (1 << 191)`. Values $\geq 2^{191}$ are negative in `i192`, and the current check misses them. While this does not result in any immediate issues, since such large values would overflow later when cast to `u64`, it is semantically wrong.

>_ *thala_oracle/sources/chainlink_oracle.move* RUST

```rust
fun parse_chainlink_price(
    benchmark: &Benchmark,
    last_price: FixedPoint64,
    staleness_seconds: u64,
    staleness_broken_seconds: u64,
    price_deviate_reject_pct: u64
): (u8, FixedPoint64) {
    [...]
    // Price must be positive
    // Chainlink encodes I192 benchmark prices as a u256 with 18 decimals.
    // Price is negative if the high bit is set. 1 << 192 checks if the highest bit set.
    if (price >= (1 << 192)) {
        // status is `broken` if price is negative
        return (status::broken_status(), fixed_point64::zero())
    };

    // Factor in 18 decimal precision
    let price_integer_component = (price / (math128::pow(10, 18) as u256) as u64);
    [...]
}
```

Remediation

Update the check by comparing the price to `(1 << 191)` instead of `(1 << 192)`.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.